Migration from On-Prem Messaging System to The Cloud: What, How and Why

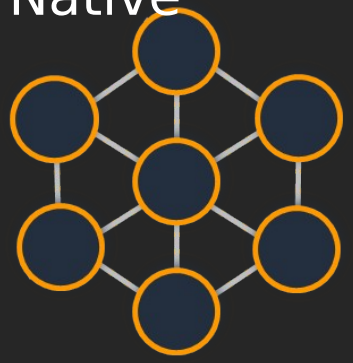
An introductory session about Messaging System Migration

Agenda

- Messaging System Overview
- Stages/milestones of migration for a messaging system
- Benefits of migrating messaging system from on-prem to serverless
- Strategy to move between the stages / key factors to consider

Messaging System Overview

Cloud Native



Microservice Design

Examples:



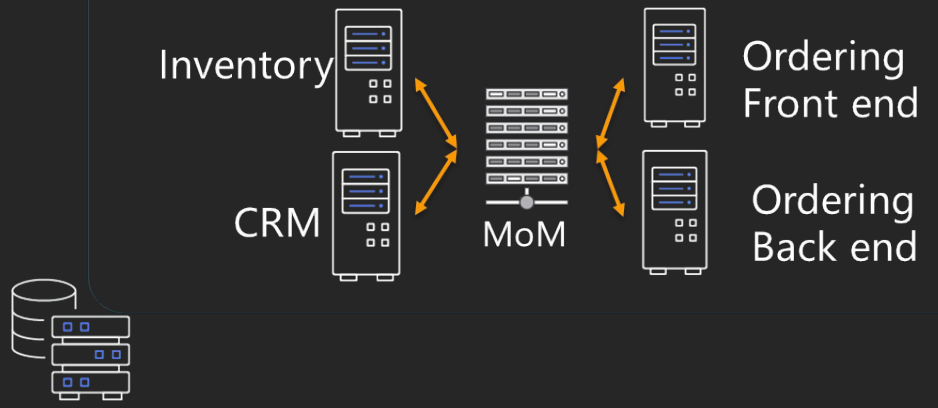
Amazon Simple Queue Services (Amazon SQS)



Amazon Simple Notification Service (Amazon SNS)

Message queuing service
Publish / subscribe messaging and notification service

Traditional Applications



Examples: IBM MQ TIBCO ActiveMQ RabbitMQ

Message-oriented middleware (MoM) or Message Broker

Messaging System Applications



Financial services
Trades and transactions



Retail
Order processing and fulfillment



Health care
Clinical data exchange



Media and entertainment
Image, video processing



Publication
Document capture and publication search



Education
Learning and engagement



Manufacturing
Multi-step process



Government
Citizen engagement

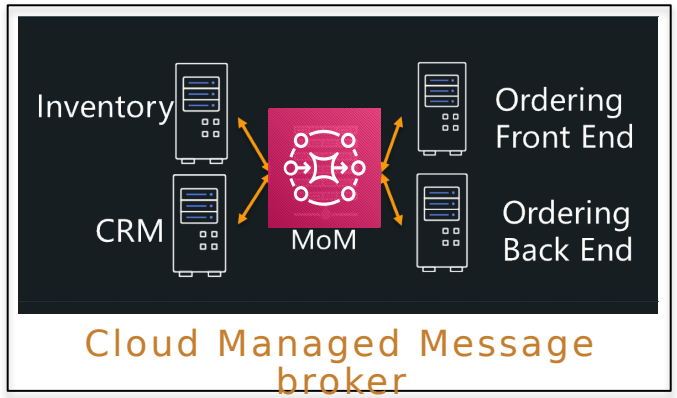
Loosely coupled, larger scale, more fault-tolerant systems

Messaging System Migration Journey

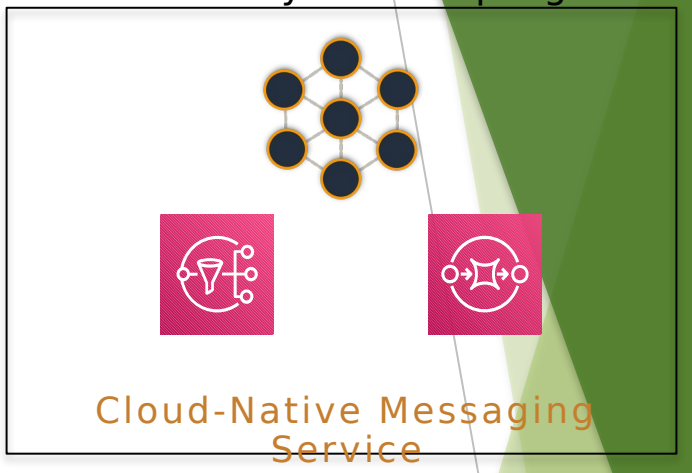
Lift and Shift, Quick Migration



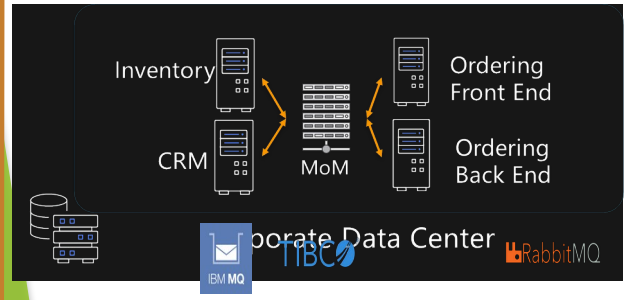
Cost Savings, Operational Efficiencies, & Availability



Higher Scalability, Reliability & Decoupling



On-premise message broker



- Expensive (Capacity and Operation Costs)
- Specialized skillset
- Infrastructure Maintenance and Management
- Difficult to scale

Responsibility Shift by Migration

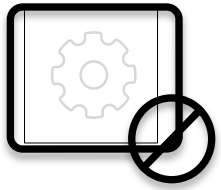
	Stages of Migration			
	ON-PREMISES	**INFRASTRUCTURE SERVICES** (i.e. EC2)	**Managed Services (i.e. Amazon MQ)**	**CLOUD NATIVE / Serverless** (SQS/SNS)
Application code	✓	✓	✓	✓
Capacity planning and scaling	✓	✓	✓	✓
Software install and maintenance	✓	✓	✓	✓
Infrastructure-Level Configuration	✓	✓	✓	✓
Physical server, storage, networking, and facilities	✓	✓	✓	✓
Security and network configuration	✓	✓ ✓	✓ ✓	✓ ✓

MANAGED BY

✓ **CUSTOMER**

✓ **Cloud Service Provider (i.e. AWS)**

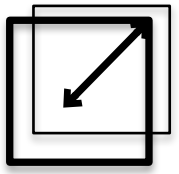
Serverless Architecture Benefits



Decouple infrastructure from business



Pay for what you use

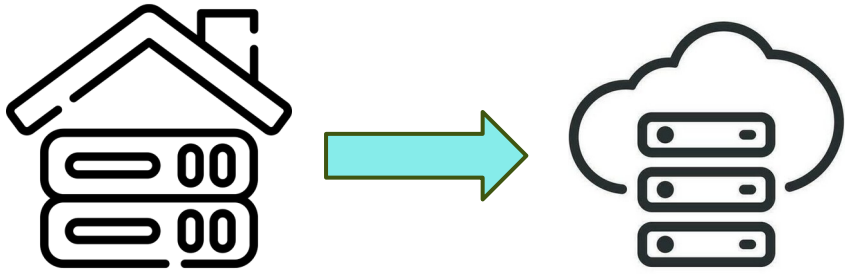


Automatically scale by unit of consumption



Highly available and durable

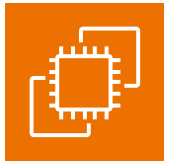
Rehost: Lift and Shift to the Cloud



On-Premise

Cloud Instance

Example:



Amazon Elastic Compute Cloud (Amazon EC2)

Pros:

- Quick and Straight Forward Migration Strategy
- Minimal changes to current application logics
- High level of customization over the Cloud instances

Cons:

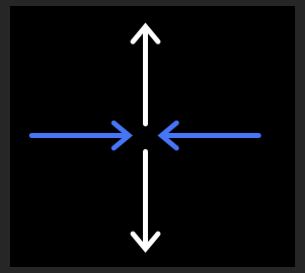
- Infrastructure and Operational tasks remain on the customer's team
- Difficulty integrating with other Cloud-Native Services
- No native built-in observability over the messaging system applications

Replatform: Migrate to Managed Services

Example: Amazon MQ for RabbitMQ



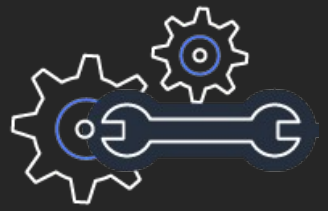
- Setup
- Automatic provisioning of single-node and clustered brokers with best practices by default



- Scaling
- Vertically scale between Amazon MQ broker instance types



- Security
- Encryption in transit over TLS and encryption at rest using KMS keys



- Upgrades
- Managed administrative tasks, such as software upgrades



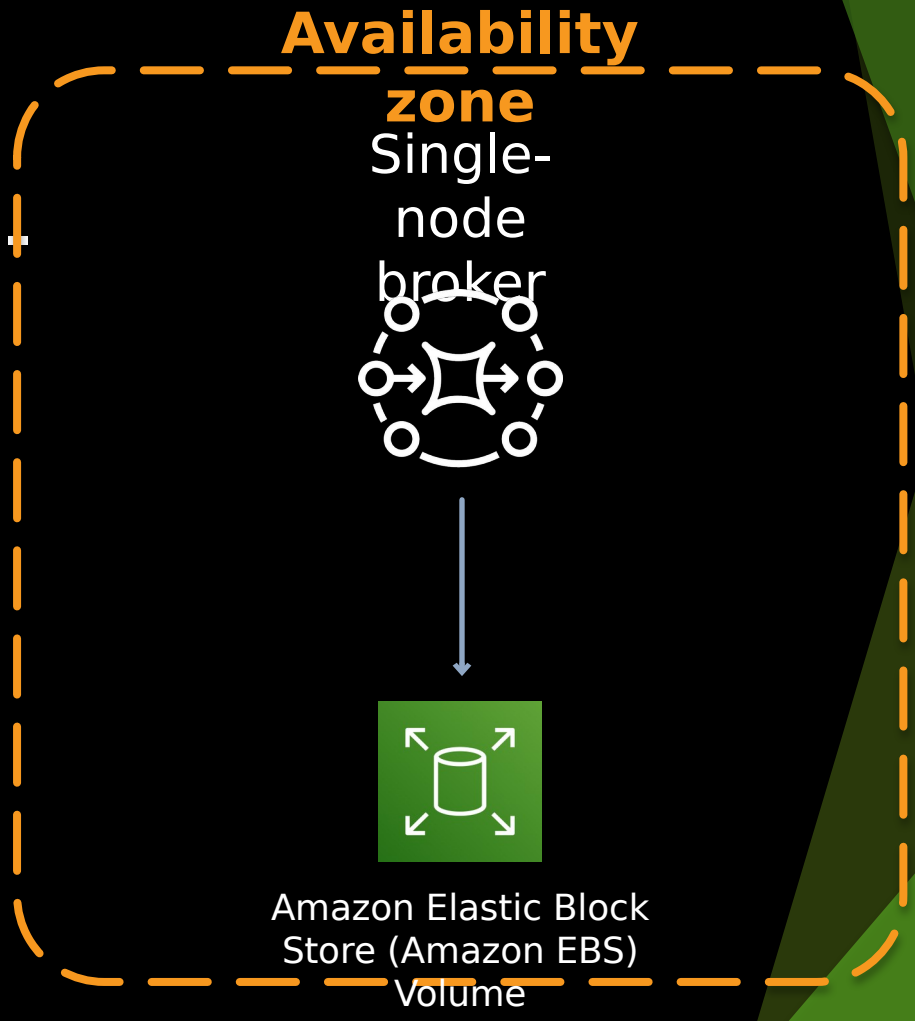
- Maintenance
- Customized maintenance window schedule

A Deeper Look into Amazon MQ for RabbitMQ

- **AMQP 0-9-1 support**
- **Choice of single-instance or three-node, clustered broker**
- **Management, Monitoring, Shovel, and Federation plugins installed**
- **AWS CloudWatch metrics for broker monitoring**
- **AWS CloudWatch Logs for broker logging**

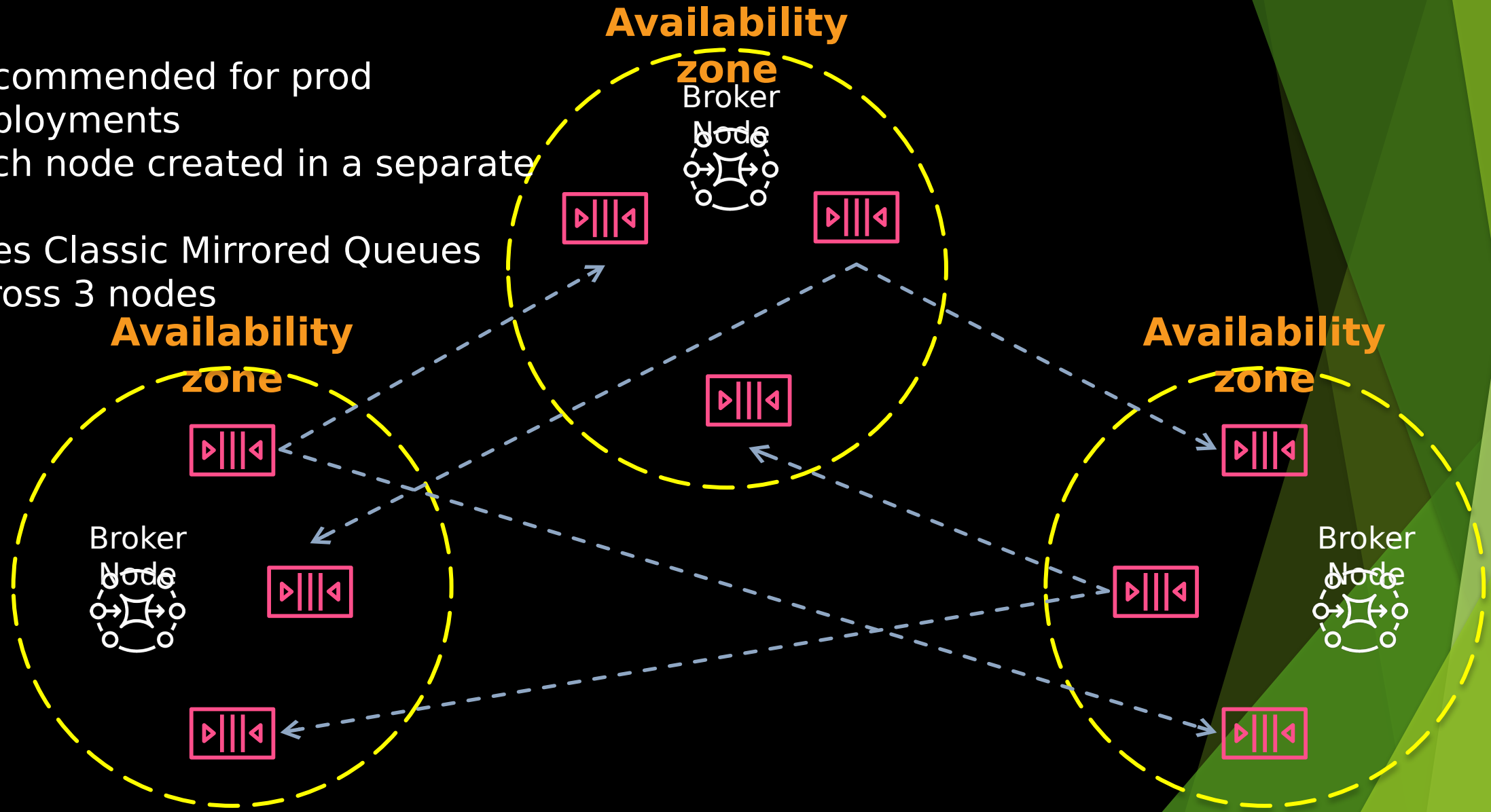
Amazon MQ for RabbitMQ Single-Node Deployment

Recommended for non-production deployments and low-latency use cases with non-persistent messages



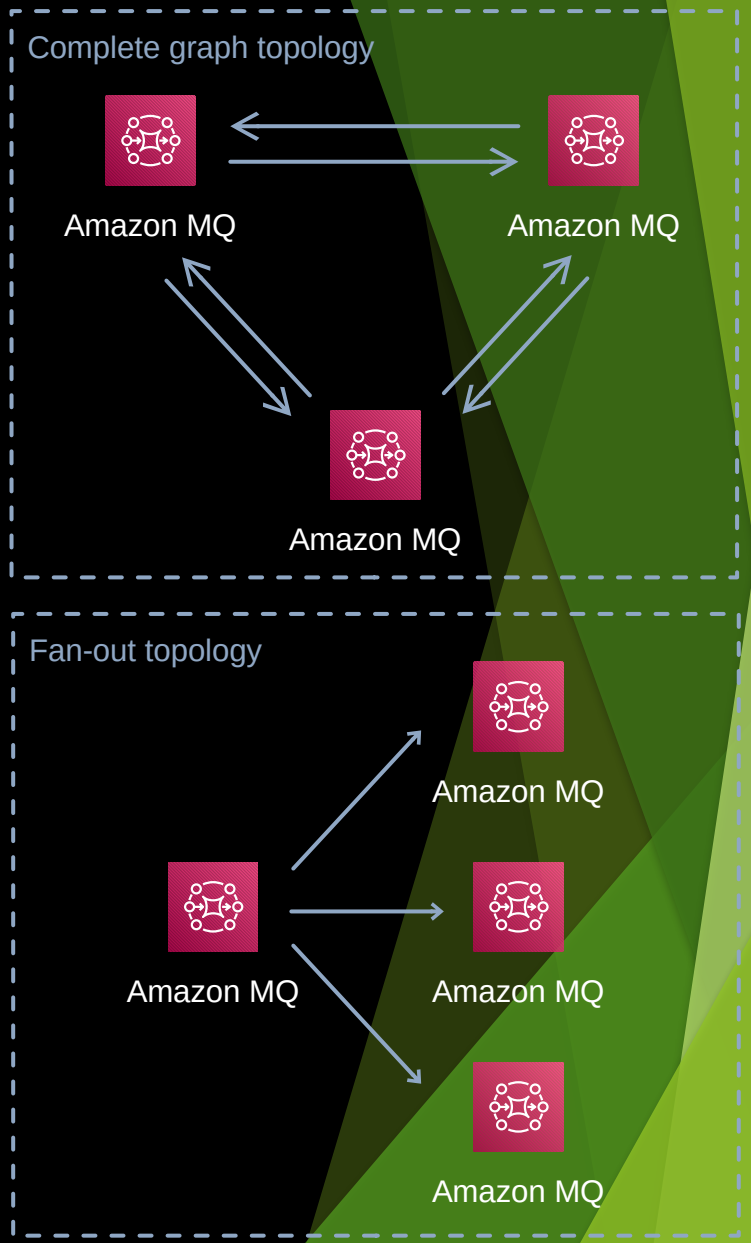
Amazon MQ for RabbitMQ Cluster Deployment

- Recommended for prod deployments
- Each node created in a separate AZ
- Uses Classic Mirrored Queues across 3 nodes



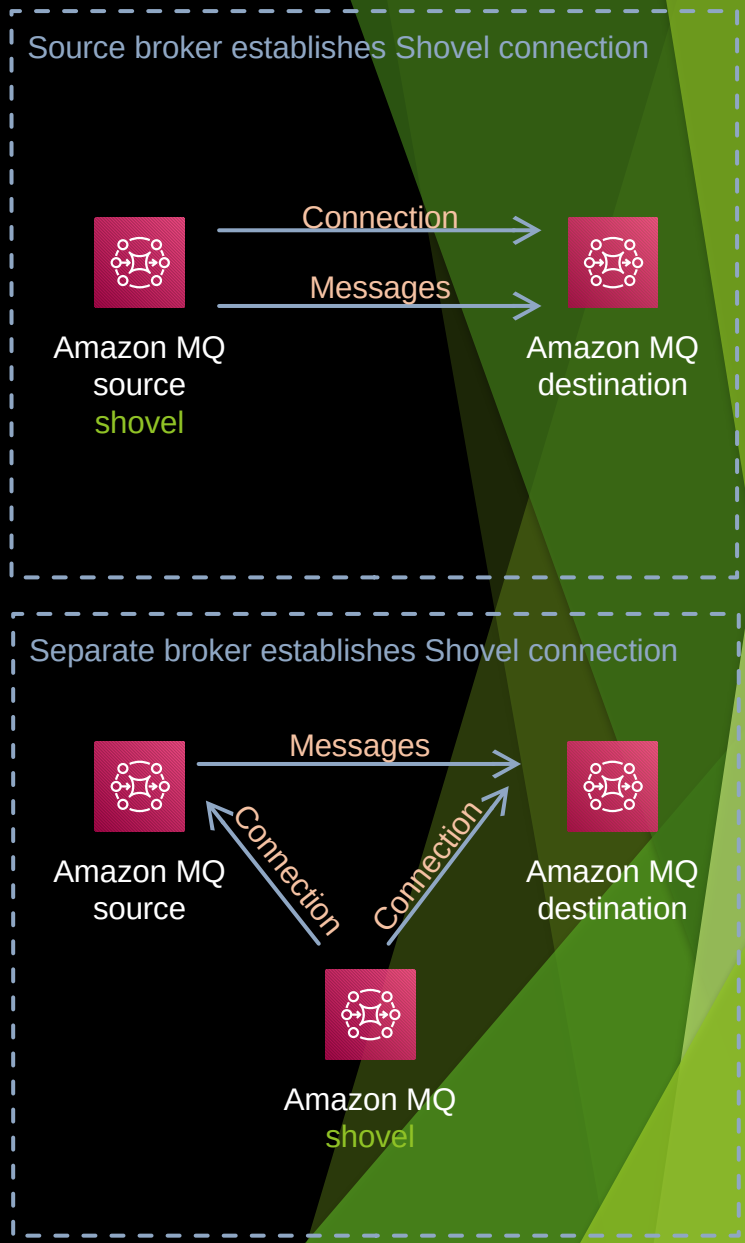
Moving Messages -- Federation

- Downstream brokers initiate one-way, point-to-point connections to upstream brokers
- Federated queues **move** messages from upstream queues to perform load balancing across downstream consumers
- Federated exchanges **copy** messages from upstream exchanges to downstream exchanges; loop protection is provided using a max-hops value



Moving Messages -- Shovel

- A shovel acts like a well-written client application, consuming messages from a queue on a source broker and moving them to a queue or exchange on a destination broker
- Shovel connections can be initiated from the source broker, destination broker, or a separate node or cluster
- After a connection is established, queues, exchanges, and bindings can be declared automatically, according to the shovel definition
- Flexible topology support, but no loop detection



Refactor: Moving into Serverless

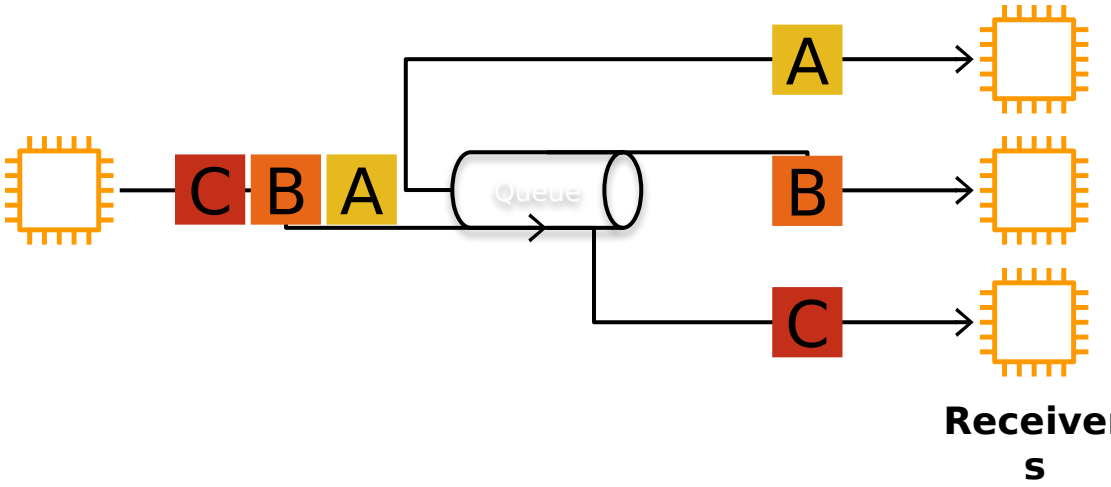
Categorize your messaging system components

- A producer publishes an event to an event router
- An event router filters and sends the events to consumers.
- A consumer receives an event from a router and processes it.
- Producer services and consumer services should be decoupled, which allows them to be scaled, updated, and deployed independently.

Refactor: Moving into Serverless

Understand message delivery model

Point-to-point (queue)

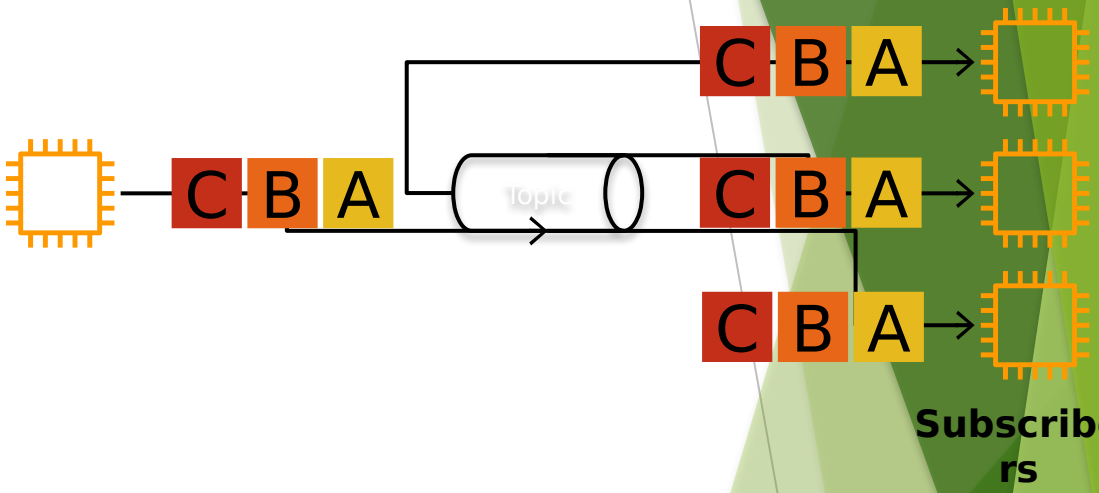


Each message consumed by one receiver

Consumers easy to scale

Buffering can flatten peak loads

Publish-subscribe (topic)



Each message consumed by each subscriber

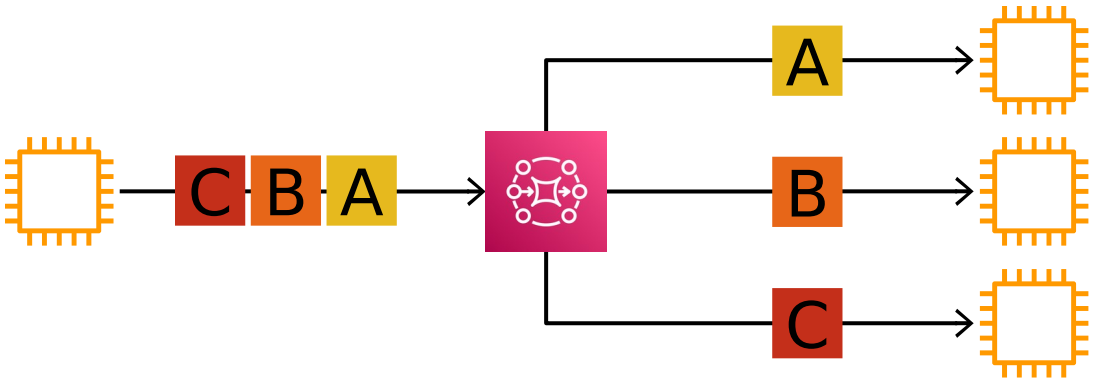
Scale consumers with new subscriptions

Conceptually no buffering

Refactor: Moving into Serverless

Replatform into Managed Service (e.g. Amazon MQ)

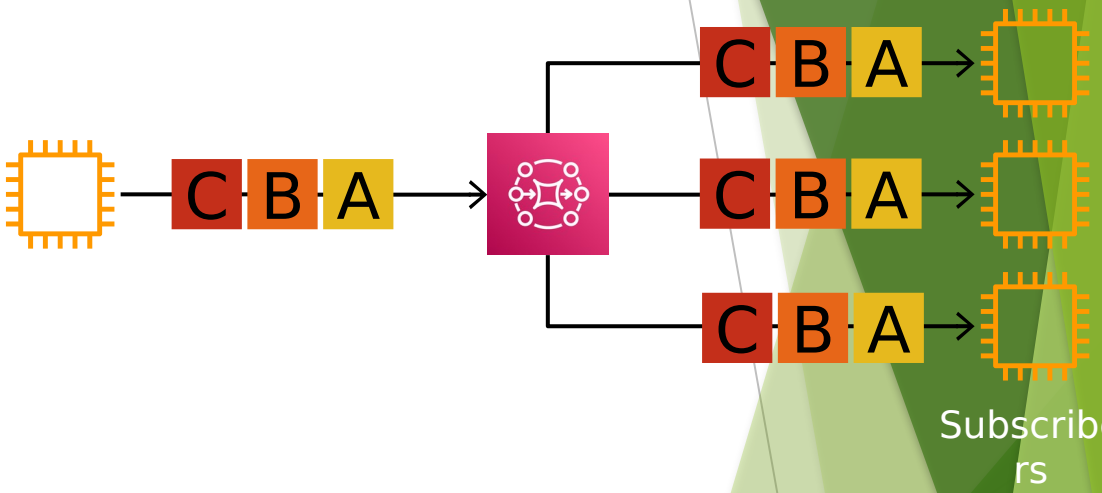
Point-to-point (queue)



Amazon MQ – managed RabbitMQ or Apache ActiveMQ

For applications constrained to industry-standard protocols like JMS or AMQP

Publish-subscribe (topic)



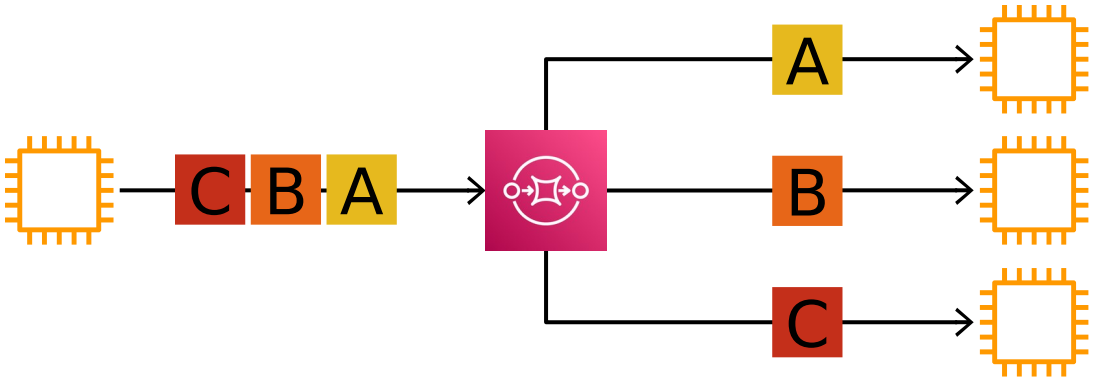
Amazon MQ – managed RabbitMQ or Apache ActiveMQ

For applications constrained to industry-standard protocols like JMS or AMQP

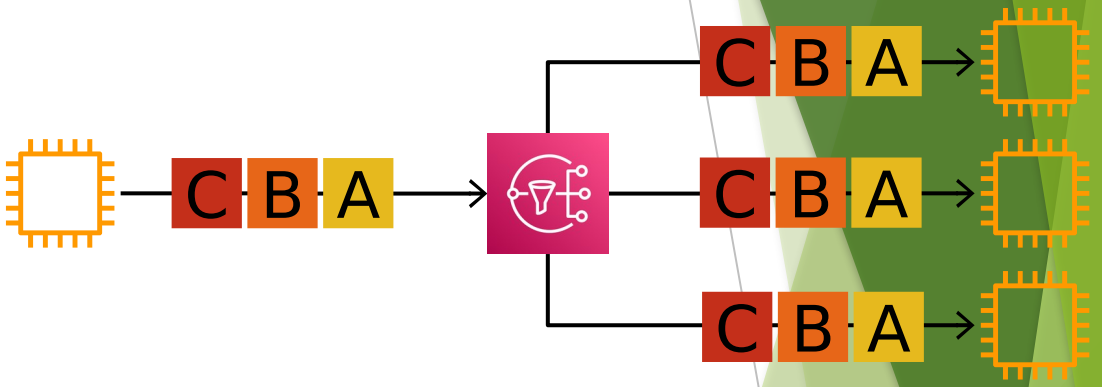
Refactor: Moving into Serverless

Refactor application logic to adopt cloud-native messaging services

Point-to-point (queue)



Publish-subscribe (topic)



Amazon Simple Queue Service (Amazon SQS)

Cloud-native & serverless

Amazon Simple Notification Service (Amazon SNS)

Cloud-native & serverless

Refactor: Moving into Serverless

Refactor Considerations

- Protocol Support – from AMQP/WebSocket/OpenWire/STOMP/MQTT to HTTPS/JMS based communication
- Message Order Preservation/FIFO: Standard vs. FIFO type of Queues/Topics
- AWS CloudTrail Logs* for Request-Level Logging
- Native integration with Authentication/Authorization through IAM/Identity Center
- Operation Efficiency – Use Temporal Queues for Request-Response Pattern / Short-Lived Queues

Key Factors to Consider...

- Nuances of each technology
- Authentication/ Authorization
- Performance testing
- Deployment
- Operations and monitoring
- Troubleshooting Mindset



In Summary:

- By migrating messaging system to the cloud, the overall architecture becomes more resilient and highly available
- There are multiple stages for migration
- A successful migration requires changes at both application level and business/operations level

